



evanexpert

FASE 2 · TECHNISCH PLAN

De architectuur, zwart op wit.

Volledige technische architectuur, datamodel, authenticatie, rollen & rechten, Mollie- en facturen-integratie, e-mail-abstractie en geïdentificeerde risico's met mitigaties.

PROJECT

Smart Buy Store

DOCUMENT

02 van 10

DATUM

14 mei 2026

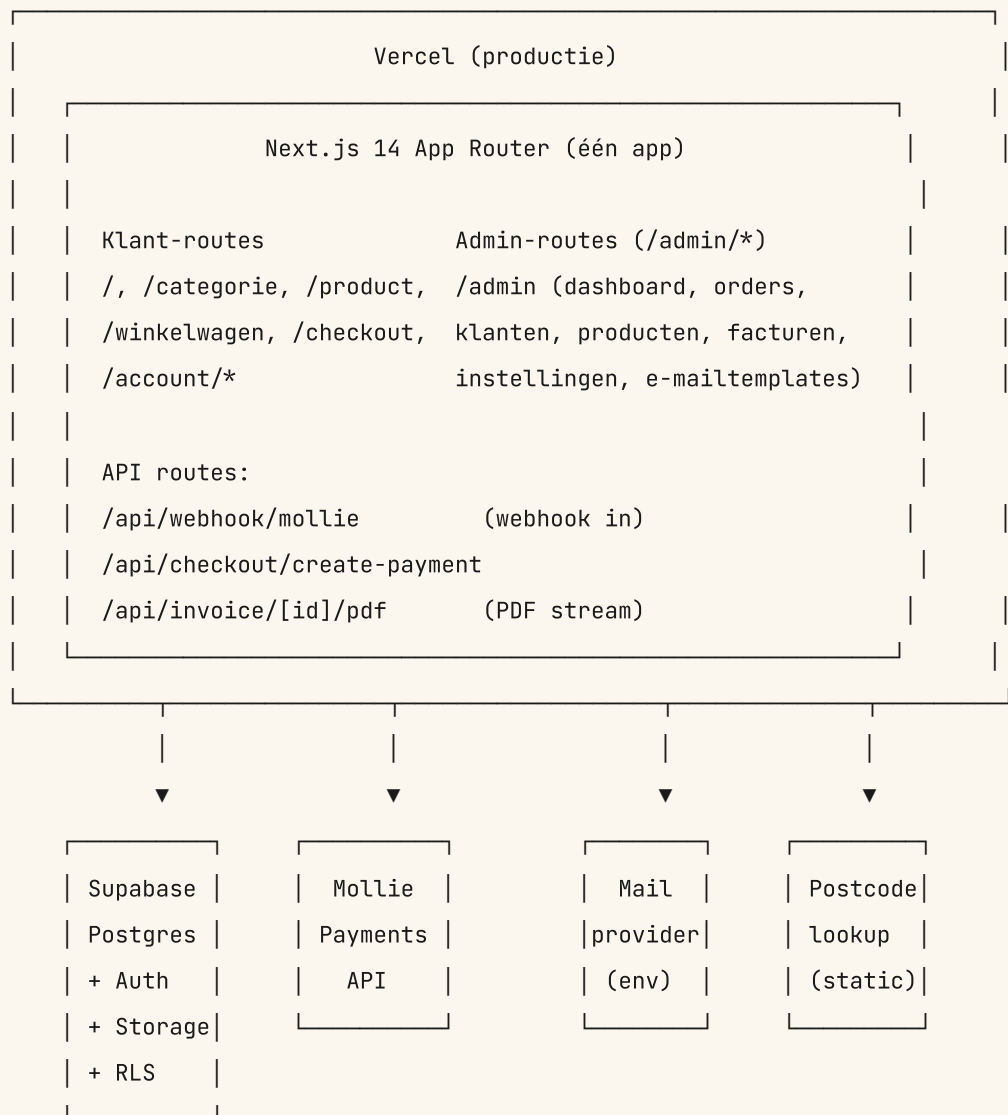
AUTEUR

Evan Poul

EvanExpert Studio · KvK 78740320 · info@evanexpert.nl

evanexpert.nl

Architectuur op één pagina



Eén Next.js app, drie externe diensten (Supabase, Mollie, Mail). De rest leeft binnen Vercel + Next.js. Geen microservices.

Stack-keuzes

Laag	Keuze	Waarom
Framework	Next.js 14 App Router	Server Components + Server Actions = geen aparte API-layer voor 90% van de calls. SSR voor SEO.
Backend / DB	Supabase (Postgres + Auth + Storage)	Eén systeem voor auth, DB en bestandsopslag. RLS = beveiliging op DB-niveau.
ORM	Drizzle ORM (TypeScript-first)	Type-safe queries, geen runtime overhead. Migraties versioned in git.
Auth	Supabase Auth (email + wachtwoord + magic link)	Wachtwoord-reset, sessies, hashing — opgelost. 2FA later mogelijk.
Betalingen	Mollie via <code>@mollie/api-client</code>	Officieel SDK. Webhook is enige bron-van-waarheid.
Mail	Resend (default) of generieke SMTP	Geabstraheerd in <code>lib/mail.ts</code> — provider wisselen = 1 env var.
PDF	<code>@react-pdf/renderer</code>	Geen Chrome headless nodig op Vercel. Templates als React-componenten.
Postcode lookup	Statische JSON (4-cijfer prefix → provincie)	Zero latency, geen API-key, geen kosten.
Hosting	Vercel	Edge-functies + ISR + Server Actions out-of-the-box.
State (client)	React state + <code>useCart</code> hook + Supabase-sync voor ingelogde users	Bestaande localStorage-cart blijft werken, sync naar DB bij login.

Datamodel

Alle tabellen onder Postgres schema `public`, met prefix `sbs_` om niet te botsen met EvanExpert n8n-tabellen in dezelfde Supabase.

```

auth.users          ← Supabase-managed (email, password hash, session)
  |
  └─ sbs_profiles    (rol, naam, telefoon, created_at)
      |
      ├── sbs_addresses (verzend- of factuuradres per gebruiker)
      └─ sbs_orders    (bestellingen, customer optioneel = guest
checkout)
          |
          ├── sbs_order_items (regels, met product-snapshot)
          ├── sbs_order_status_log (audit trail)
          ├── sbs_payments    (Mollie payment records, history)
          └─ sbs_invoices     (immutable snapshot, PDF in Storage)

sbs_brands — sbs_products — sbs_categories
          |
          └─ sbs_product_images, sbs_product_specs

sbs_discount_codes
sbs_email_templates
sbs_email_log
sbs_mollie_events
sbs_settings (key-value, JSONB)
sbs_postcodes_nl (static reference)
  
```

Belangrijkste tabellen

sbs_profiles — gekoppeld 1-op-1 met `auth.users.id`

```

id (uuid, FK auth.users), role (enum: admin|staff|delivery|customer),
full_name, phone, is_active, created_at, updated_at
  
```

sbs_orders — guest checkout toegestaan (`user_id` nullable)

```
id (uuid), order_number (text, uniek, oplopend),
user_id (nullable FK profiles),
status (enum: pending_payment | paid | in_progress | planned_delivery |
        delivered | completed | cancelled | refunded),
delivery_method (enum: standard | same_day),
delivery_date (date), delivery_postcode (text),

customer_snapshot (jsonb: email, name, phone),
shipping_address_snapshot (jsonb),
billing_address_snapshot (jsonb),

subtotal_excl_btw, btw_total, discount_amount, total_incl_btw (numeric),
discount_code (text, snapshot van de code),
notes_customer (text), notes_internal (text),
created_at, updated_at
```

sbs_order_items — product-snapshot zodat prijswijzigingen geen oude orders raken

```
id, order_id (FK), product_id (FK, nullable als product later verwijderd),
product_snapshot (jsonb: name, brand, sku, image),
qty (int), unit_price_excl_btw, btw_rate (default 21),
line_subtotal_excl_btw, line_btw, line_total_incl_btw
```

sbs_payments — historisch logboek

```
id, order_id (FK), mollie_payment_id (uniek),
status (mirror van Mollie), amount, method,
raw (jsonb dump van Mollie response),
created_at, paid_at
```

sbs_mollie_events — idempotentie

```
event_id (uuid, PK), payment_id, order_id,
payload (jsonb), received_at, processed_at,
result (enum: ok | duplicate | error)
```

sbs_invoices — onveranderlijk

```
id (uuid), invoice_number (text, uniek oplopend per jaar: SBS-2026-0001),
order_id (FK uniek),

customer_snapshot (jsonb),
company_snapshot (jsonb: naam, adres, KvK, BTW-nummer),
items_snapshot (jsonb array),
totals_snapshot (jsonb: subtotaal, BTW per tarief, total),

pdf_path (text, pad in Supabase Storage),
generated_at (timestamp),

-- REVOKE UPDATE on this table – DB-level beveiliging
```

Sequence `sbs_invoice_seq_2026` genereert het volgnummer atomically — geen botsing bij gelijktijdige betalingen.

sbs_email_log — audit + idempotentie

```
id, order_id, customer_email, event_type,
status (enum: queued|sent|failed), provider_message_id,
sent_at, error_message,
UNIQUE (order_id, event_type) ← DB-level idempotentie
```

Authenticatie

Eén auth-systeem voor klant én admin via Supabase Auth. Verschil zit in `sbs_profiles.role`.

Klant — flow

1. Klant maakt account aan via `/account/register` met email + wachtwoord
2. Supabase verstuurt verificatie-mail (via onze mail-laag)
3. Klant klikt link → email is geverifieerd → `sbs_profiles` row aangemaakt met role `customer`
4. Inloggen via `/account/login` — Supabase session cookie (httpOnly, secure, SameSite=lax)
5. Server Components lezen de session via `@supabase/ssr`
6. **"Bestellingen claimen" flow**: oude bestellingen waar `user_id IS NULL` maar `customer_snapshot.email = user.email` worden bij eerste login automatisch gekoppeld

Admin / Medewerker / Bezorger — flow

- Worden aangemaakt door bestaande admin in `/admin/accounts`
- Krijgen invite-link voor wachtwoord-aanmaak
- Geen self-registration voor admin-rollen
- Login via dezelfde `/account/login` of `/admin/login` (zelfde flow)
- Middleware redirect role admin/staff/delivery naar `/admin`, customer naar `/account`

Sessions & middleware

```
middleware.ts:
- /admin/* → check session, check role IN (admin, staff, delivery)
- /account/* → check session, redirect naar /account/login als geen session
- /api/webhook/* → géén auth check (webhooks zijn unsigned; security via refetch)
- /api/admin/* → check role IN (admin, staff) server-side
```


Rollen en rechten

Vier rollen, geënfonceerd op **drie niveaus**: middleware, server-side checks, en RLS.

Rol	Mag zien	Mag wijzigen	Mag niet
admin	Alles	Alles	Zichzelf demoten als laatste admin
staff	Bestellingen, klanten, producten, categorieën	Producten, bestelstatussen, klantgegevens	Instellingen, Mollie-key, andere accounts, kortingscodes >25%, e-mailtemplates
delivery	Alleen bestellingen <code>paid</code> / <code>in_progress</code> / <code>planned_delivery</code>	Alleen <code>delivery_status</code> veld	Al het andere
customer	Eigen profiel, eigen bestellingen, eigen facturen	Eigen profiel + adressen	Andermans data, admin-routes

RLS-voorbeelden

```
-- Klanten zien alleen eigen bestellingen
create policy "customer sees own orders"
on sbs_orders for select
to authenticated
using (
  auth.uid() = user_id
  or
  (select role from sbs_profiles where id = auth.uid()) in ('admin','staff','delivery')
);

-- Niemand kan invoice-rijen UPDATE'n
revoke update on sbs_invoices from authenticated, anon, service_role;
```

"Laatste admin"-bescherming

DB-trigger op `sbs_profiles` : voorkomt dat de laatste actieve admin gedemoot of gedeactiveerd wordt.

Mollie-integratie

Setup (FASE 7 stap 0)

1. Smart Buy registreert Mollie-account (test-mode start)
2. API-key in Vercel als env: `MOLLIE_API_KEY` (test_xxx of live_xxx)
3. Webhook URL in Mollie dashboard: `https://smartbuystore.nl/api/webhook/mollie`

Betalingsflow (chronologisch)

1. Klant klikt "Betalen" in checkout
↓
 2. Server Action: createOrder({...})
 - Maakt sbs_orders row status='pending_payment'
 - Genereert order_number via DB sequence
 - Maakt Mollie Payment met returnUrl + webhookUrl
 - Slaat sbs_payments row op met mollie_payment_id
 ↓
 3. Server retournt mollie.checkoutUrl
↓
 4. Client redirect naar Mollie checkout
↓
 5. Klant betaalt
↓
 - 6a. Klant terug naar /checkout/return?order=SBS-2026-XXXX
 - "We controleren je betaling..." loader
 - Polled getOrderStatus elke 2 sec, max 30 sec
 - Veranderd NOOIT status zelf – leest alleen DB
 ↓
 - 6b. PARALLEL: Mollie POSTs naar /api/webhook/mollie
Body: { id: 'tr_xxxxx' }
- Handler:
- a) Refetch payment via mollie.payments.get(tr_xxxxx) – NEVER trust body
 - b) INSERT INTO sbs_mollie_events – als duplicate → return 200
 - c) Get sbs_orders WHERE id = metadata.order_id
 - d) If payment.status == 'paid' && order.status == 'pending_payment':
 - UPDATE sbs_orders status='paid'
 - INSERT sbs_payments status='paid'
 - INSERT sbs_invoices (snapshot + PDF gen queued)
 - INSERT sbs_email_log (order_id, 'payment_received') ON CONFLICT DO NOTHING
 - Trigger mail send als email_log insert succeeded
 - LOG to sbs_order_status_log
 - e) Return 200 OK
- ↓
7. Volgende poll detecteert order.status='paid' → toont bevestigingspagina

Wat dit waarborgt: - Status `paid` ontstaat alleen via webhook - URL manipulation door klant kan niets uitrichten - Dubbele webhook → idempotent via `sbs_mollie_events` - Dubbele factuur →

unieke index op `sbs_invoices.order_id` - Dubbele e-mail → unieke index op `sbs_email_log`
(`order_id`, `event_type`)

Facturen — snapshot pattern

Trigger: alleen wanneer order van `pending_payment` → `paid` overgaat.

Snapshot omvat: - Klantgegevens (naam, email, adres) uit `customer_snapshot` - Bedrijfsgegevens (Smart Buy naam, adres, KvK, BTW-nummer) uit `sbs_settings` - Regels (per regel: naam, aantal, prijs, BTW-tarief, BTW-bedrag, totaal) - Totalen - Factuurnummer (sequentieel uit DB-sequence) - Factuurdatum (= betaaldatum)

PDF-rendering via `@react-pdf/renderer` — direct in Node, geen browser nodig. Upload naar Supabase Storage `invoices/`, signed URL voor klant-toegang.

Onveranderbaar — drie niveaus: 1. `REVOKE UPDATE ON sbs_invoices FROM authenticated, service_role` 2. Geen admin-UI om factuurregels te bewerken 3. Storage bucket is `public: false` — alleen via signed URLs

E-mail — abstractie + idempotentie

Mail-laag (`lib/mail.ts`)

```
sendMail({
  to: 'klant@example.nl',
  subject: '...',
  templateSlug: 'payment_received',
  context: { orderNumber, customerName, ... },
  attachments: [{ filename: 'factuur.pdf', content: Buffer }]
})
```

Provider geselecteerd via env: - `MAIL_PROVIDER=resend` → Resend SDK - `MAIL_PROVIDER=smtp` → nodemailer met SMTP-credentials

Templates in DB (`sbs_email_templates`)

```
slug PK (e.g. 'order_received', 'payment_received', ...),
subject, body_html, body_text,
updated_by, updated_at, version
```

Admin kan templates aanpassen in `/admin/e-mailtemplates`. Variabelen via `{{customerName}}` - syntax.

Idempotentie

```
INSERT INTO sbs_email_log (order_id, event_type, status)
VALUES ($1, $2, 'queued')
ON CONFLICT (order_id, event_type) DO NOTHING
RETURNING id;
```

- Geen RETURNING → al verstuurd, skip
- Anders → verstuur, UPDATE status naar 'sent' of 'failed'

Welke mails

Trigger	Template slug	Bevat factuur PDF?
Order aangemaakt	order_received	nee
Order paid (vanuit webhook)	payment_received	ja
Status → in_progress	order_in_progress	nee
Status → planned_delivery	order_planned_delivery	nee
Status → delivered	order_delivered	nee
Status → cancelled	order_cancelled	nee

Risico's & mitigaties

Risico	Impact	Mitigatie
Webhook arriveert voor klant op <code>/checkout/return</code> is	Klant ziet "even wachten"	UI polled DB elke 2s, nette wachtmelding.
Webhook arriveert te laat / niet	Bestelling blijft <code>pending_payment</code>	Cron <code>/api/jobs/reconcile-payments</code> elke 15 min — refetcht status bij Mollie.
Stock overselling	Klant betaalt voor niet-leverbaar product	v1: stock is informational. v2: optimistic lock. Admin UI heeft "Annuleren + terugbetalen" knop.
Factuurnummer-collision	Wettelijk probleem	DB-sequence atomisch — geen race condition.
Test- vs live-key gewisseld	Test-betalingen in productie	Env-vars per environment (Vercel Preview/Production).
Admin demoteert zichzelf, geen admin meer over	Geen toegang meer	DB-trigger: laatste actieve admin niet demoot-baar.
RLS-policy bug → klant ziet andermans data	Privacy-incident	RLS unit-tests in CI. Defense-in-depth: server-side filters op <code>user_id</code> .
E-mail in spam	Klanten ervaren "geen bevestiging"	SPF + DKIM + DMARC via Resend. Custom from-domain <code>noreply@smartbuystore.nl</code> .
API-key lekt	Mollie-account compromise	Server-side only, nooit in <code>NEXT_PUBLIC_*</code> . Linter-check.
Snapshot mist BTW-nummer	Wettelijk probleem	Snapshot is geheel, oude facturen behouden oud BTW-nummer. UPDATE revoked.
Webhook DoS / replay	Server-overload	IP-whitelist optioneel. Idempotentie zorgt dat replay geen schade doet.
Vercel function timeout op PDF-gen	Webhook faalt	<code>@react-pdf/renderer</code> < 1s typisch. Mitigatie: PDF als background-job NA webhook-200.
Klant maakt nieuw account met andere email	Bestellingen niet zichtbaar	"Claim by email" flow met mail-verificatie.

Risico	Impact	Mitigatie
Source code raakt weer kwijt	Geen versie-controle	FASE 4 stap 1: GitHub-repo + git initial commit + push. CI dwingt af.

Akkoord op deze fase ontvangen. Doorgegaan naar FASE 3 — UX & schermen.